AuraMart

New Laptop Setup Guide

Download, install and verify the complete tech stack on Windows 11

Companion files: AuraMart\2-laptop-setup · versions checked 15 Oct 2026

At a glance

For	A fresh **Windows 11** laptop (64-bit). Sign in with your own account, which needs admin rights.
Result	Every tool AuraMart uses, from Sprint 0 to Sprint 13, installed, configured and checked.
Your time	About 1 hour with the install script (Path A), or 2–3 hours step by step (Path B), plus download time.
Downloads	About 5 GB to finish Sprint 0; roughly 15 GB for the whole project, including the AI models (approximate).
Disk	40 GB free recommended (check with the script in Section 7).
Memory	16 GB RAM recommended. 8 GB works with the limits in Section 9.
Companion files	The **AuraMart\2-laptop-setup** folder — install script, check script, pre-download script and the config files this guide uses.

Choose your path

Path	Best when	Where
A — one script	You want it done fast. One command installs every tool with winget; you then restart and sign in.	Section 3
B — step by step	You want to learn what each tool does, or the script hit a problem.	Section 4

> **Then**
>
> Both paths end in the same place: Section 5 puts the AuraMart code on the laptop, Section 7 checks everything. If you already set up one laptop, Section 10 explains how to work across two.

Contents

1. Before you start **3**

2. What you will install **4**

3. Path A — install everything with one script **6**

4. Path B — install step by step **7**

Step 1 · Prepare Windows 11 . . . 7

Step 2 · Git and GitHub CLI . . . 8

Step 3 · Java 21 (Eclipse Temurin) . . . 8

Step 4 · Apache Maven 3.9 . . . 8

Step 5 · Docker Desktop . . . 8

Step 6 · IntelliJ IDEA and VS Code . . . 9

Step 7 · Node.js LTS . . . 9

Step 8 · Helper apps . . . 9

Step 9 · AI and Kubernetes tools (when you reach them) . . . 9

5. Get the AuraMart code onto the laptop **10**

6. Download everything up front (optional) **11**

7. Start everything and verify **12**

8. Configure the IDE, secrets and ports **13**

9. Memory and performance **14**

10. Working across two laptops **16**

11. Troubleshooting **17**

12. Daily cheat sheet **18**

Appendix A. macOS and Linux equivalents **19**

Appendix B. What's in the 2-laptop-setup folder **19**

1. Before you start

Five minutes of checks now saves an hour of confusing errors later.

Check	How	You need
Windows version	Press Win + R, type winver, press Enter	Windows 11, fully updated (Settings → Windows Update)
RAM	Settings → System → About → Installed RAM	16 GB recommended; 8 GB minimum
Free disk	File Explorer → This PC	40 GB free on C:
Virtualization	Task Manager → Performance → CPU → “Virtualization”	Enabled. If Disabled, turn on Intel VT-x / AMD-V (SVM) in the BIOS/UEFI settings
Admin rights	Right-click Start → Terminal (Admin) opens without asking for another account	Your own account must be an administrator
winget	In a terminal: winget --version	A version number. If missing: Microsoft Store → App Installer → Update
GitHub account	github.com	Signed in, with the email you will use for commits

On a company laptop?

Docker Desktop's licence is free for personal use, education and small businesses, but large companies need a paid subscription — ask IT before installing it on a work laptop. **Office networks** often block or inspect downloads (proxies, SSL inspection), which breaks Maven and Docker in confusing ways. If you can, set up on a home network; otherwise see the proxy row in Section 11.

Decide where the code lives

- Use **C:\dev\auramart** — short, no spaces.
- Do **not** put the project in Documents, Desktop or any OneDrive folder: syncing thousands of build files slows everything down and causes file-lock errors.
- Unzip **AuraMart.zip** into Downloads: this guide assumes the folder **Downloads\AuraMart**, the one that contains START-HERE.txt. (With File Explorer's Extract All, change the suggested folder to Downloads, or you get AuraMart inside AuraMart.)

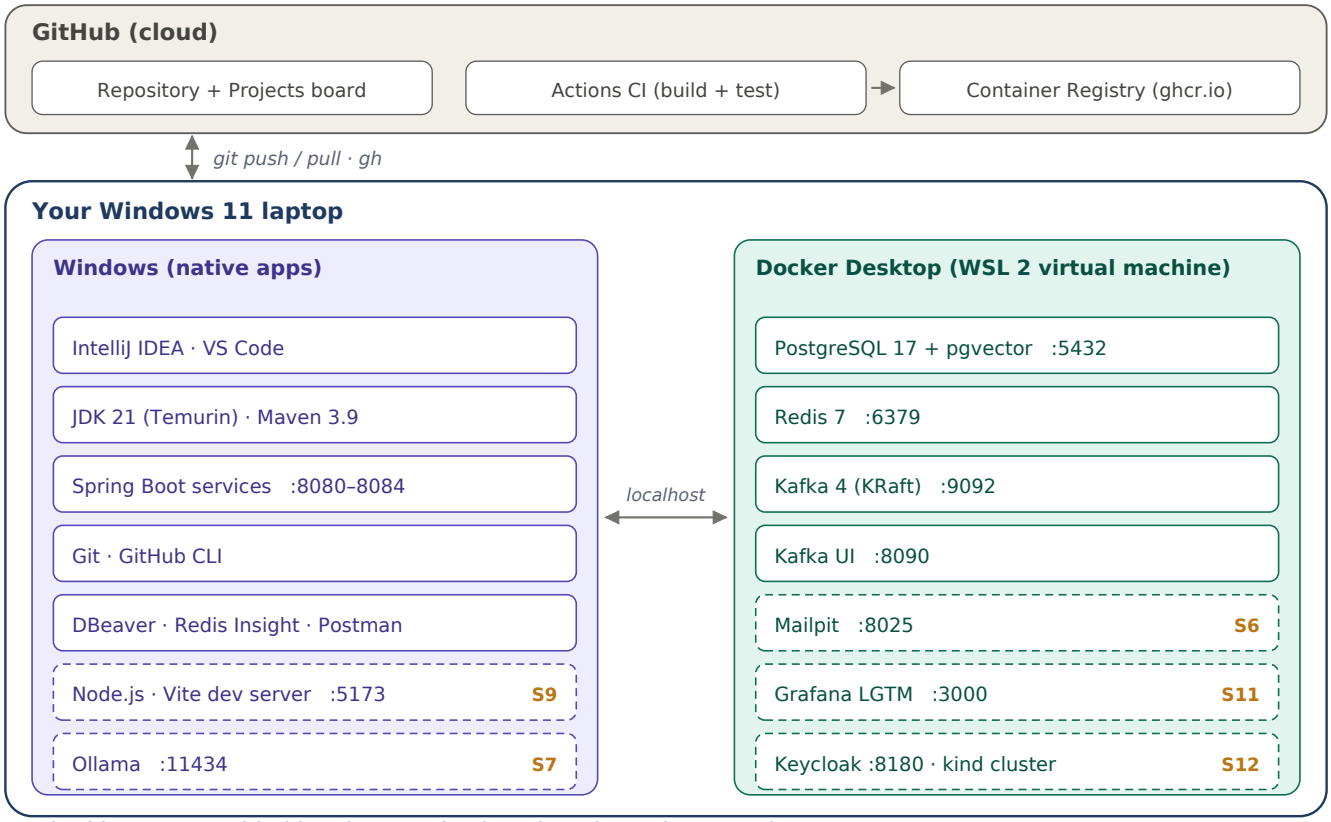
How to open the right terminal

You need	Do this
Normal terminal	Right-click the Start button → Terminal
Admin terminal	Right-click the Start button → Terminal (Admin) → Yes
A fresh PATH after installing something	Close every terminal window and open a new one (installers change PATH; open windows don't see it)

Commands in this guide are for PowerShell, the default in Windows Terminal. Lines starting with # are comments.

2. What you will install

Your code and tools run directly on Windows. The infrastructure (database, cache, message broker) runs in Docker containers inside a small Linux virtual machine (WSL 2). They talk to each other through localhost ports. GitHub holds the code, the task board, CI and the container images.



Dashed boxes are added in a later sprint (number shown in orange).

The complete download list

Tool	Version	Official download	winget ID	Size*	From
WSL 2	latest	Built into Windows: wsl --install	–	0.2 GB	S0
Git for Windows	latest	git-scm.com/downloads/win	`Git.Git`	70 MB	S0
GitHub CLI	latest	cli.github.com	`GitHub.cli`	15 MB	S0
Java JDK (Eclipse Temurin)	21 LTS	adoptium.net/temurin/releases/?version=21	`EclipseAdoptium.Temurin.21.JDK`	180 MB	S0
Apache Maven	3.9.x	maven.apache.org/download.cgi	– (zip)	10 MB	S0
Docker Desktop	latest	docker.com/products/docker-desktop	`Docker.DockerDesktop`	600 MB	S0
JetBrains Toolbox → IntelliJ IDEA	2026.x	jetbrains.com/toolbox-app	`JetBrains.Toolbox`	1.3 GB	S0
Visual Studio Code	latest	code.visualstudio.com/download	`Microsoft.VisualStudioCode`	0.4 GB*	S0

Tool	Version	Official download	winget ID	Size*	From
PowerShell 7 (recommended)	7.x	learn.microsoft.com/powershell	`Microsoft.PowerShell`	110 MB	S0
Node.js	LTS	nodejs.org/en/download	`OpenJS.NodeJS.LTS`	30 MB	S9
DBeaver Community	latest	dbeaver.io/download	`DBeaver.DBeaver.Community`	120 MB	S1
Redis Insight	latest	redis.io/insight	`RedisInsight.RedisInsight`	150 MB	S3
Postman (optional)	latest	postman.com/downloads	`Postman.Postman`	150 MB	any
Ollama	latest	ollama.com/download	`Ollama.Ollama`	1 GB	S7
kind	latest	kind.sigs.k8s.io	`Kubernetes.kind`	10 MB	S12
kubectl	latest	kubernetes.io/docs/tasks/tools	`Kubernetes.kubectl`	50 MB	S12
k6	latest	grafana.com/docs/k6	`GrafanaLabs.k6`	50 MB	S12

** Sizes are rough and change between versions. VS Code includes the extensions in Step 6. "From" = the sprint that first needs it (S0 = Sprint 0).*

Downloaded for you by Docker, Maven, npm and Ollama

What	Name	Size*	From
Container image	`pgvector/pgvector:pg17 (PostgreSQL 17 + pgvector)`	0.4 GB	S0
Container image	`redis:7.4-alpine`	50 MB	S0
Container image	`apache/kafka:4.1.2`	0.4 GB	S0
Container image	`ghcr.io/kafbat/kafka-ui:latest`	0.3 GB	S0
Container image	`axllent/mailpit:latest`	30 MB	S6
Container image	`grafana/otel-lgtm:latest`	1.5 GB	S11
Container image	`quay.io/keycloak/keycloak:26.8.1`	0.5 GB	S12
Container image	`kindest/node (pulled by kind create cluster)`	0.5 GB	S12
AI model	`nomic-embed-text (embeddings)`	0.3 GB	S7
AI model	`qwen3:8b — or qwen3:4b on laptops with 16 GB RAM or less`	5 GB / 2.5 GB	S8
Java libraries	`Spring Boot, Spring AI, Kafka client, Testcontainers… (Maven cache in %USERPROFILE%\.m2)`	0.5–1 GB	S0–S12
npm packages	`React, Vite, Tailwind, TanStack Query (node_modules)`	0.3 GB	S9

3. Path A — install everything with one script

The script installs every tool in the download list with winget, puts Maven in C:\tools, sets JAVA_HOME and MAVEN_HOME, enables WSL 2 and long file paths, allows PowerShell scripts for your user, configures Git and adds the VS Code extensions. Running it twice is safe: tools that are already installed are skipped.

Run it

PowerShell

```
# 1. Unzip AuraMart.zip into Downloads -> you get Downloads\AuraMart (normal or admin terminal)
Expand-Archive "$env:USERPROFILE\Downloads\AuraMart.zip" `
    -DestinationPath "$env:USERPROFILE\Downloads"

# 2. In Terminal (Admin):
cd "$env:USERPROFILE\Downloads\AuraMart\2-laptop-setup"
powershell -ExecutionPolicy Bypass -File .\scripts\install-tools.ps1

#    ...or include the Sprint 12 tools (kind, kubectl, k6) as well:
powershell -ExecutionPolicy Bypass -File .\scripts\install-tools.ps1 -IncludeLater
```

`-ExecutionPolicy Bypass` lets this one script run without changing anything permanently. The script asks for your name and email for Git commits near the end. It takes 20–60 minutes, mostly downloading.

After the script

1. **Restart Windows.** WSL 2 and Docker Desktop need it.
2. **Start Docker Desktop** from the Start menu, accept the terms, skip the sign-in. Wait until it says the engine is running.
3. **Open JetBrains Toolbox** (system tray) and install **IntelliJ IDEA**. The free edition is enough for this project.
4. **Sign in to GitHub** in a new terminal: `gh auth login` → GitHub.com → HTTPS → Login with a web browser.
5. **Check the laptop:** `powershell -ExecutionPolicy Bypass -File .\scripts\check-env.ps1` (from the same folder).
6. **Copy your .wslconfig** to set a memory limit for Docker (Section 9).

Then continue with Section 5. If any tool shows FAILED or [!!], install that one with the matching step in Section 4.

What the check script prints

Example output (shortened)

```
AuraMart environment check
==========================
Laptop
  [ OK ]  RAM                          16 GB (16 GB recommended, 8 GB minimum)
  [ OK ]  Free space on C:             212 GB free (40 GB recommended)
  [ OK ]  Long file paths
  [ OK ]  Script policy (for npm)      RemoteSigned
Core tools (Sprint 0)
  [ OK ]  Git                          git version 2.x
  [ OK ]  Java 21                      openjdk version "21.0.x" ...
  [ !! ]  Docker engine running        engine            <- start Docker Desktop
  ...
Later sprints
  [ -- ]  kind (Sprint 12)             not installed yet
```

4. Path B — install step by step

Each step installs one part, configures it and checks it. The winget commands do the same as the script; the official download links in Section 2 work too if you prefer installers.

> **Golden rule**
>
> Every time a step installs something, **close the terminal and open a new one** before the verify commands — otherwise Windows hasn't updated PATH in that window and you'll see “not recognized” errors.

Step 1 · Prepare Windows 11

In **Terminal (Admin)**:

```
# Install WSL 2 (the Linux layer Docker Desktop runs on), then RESTART Windows
wsl --install --no-distribution

# Allow file paths longer than 260 characters (node_modules needs this)
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem" `
    -Name "LongPathsEnabled" -Value 1 -PropertyType DWORD -Force

# Optional but recommended: PowerShell 7 (supports && and has better defaults)
winget install --id Microsoft.PowerShell -e
```

Then in a **normal** terminal (after the restart):

```
wsl --version                                        # shows the WSL version
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned  # lets npm and project scripts run
```

Step 2 · Git and GitHub CLI

```
winget install --id Git.Git -e
winget install --id GitHub.cli -e

# --- new terminal ---
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"   # same email as your GitHub account
git config --global init.defaultBranch main
git config --global core.longpaths true

gh auth login      # GitHub.com -> HTTPS -> Login with a web browser
gh auth status     # should say: Logged in to github.com
```

Tip: to keep your real email private, use the no-reply address from GitHub → Settings → Emails.

Step 3 · Java 21 (Eclipse Temurin)

```
winget install --id EclipseAdoptium.Temurin.21.JDK -e

# --- new terminal: point JAVA_HOME at it ---
$jdk = Get-ChildItem "C:\Program Files\Eclipse Adoptium" -Directory |
       Where-Object Name -like "jdk-21*" | Sort-Object Name -Descending | Select-Object -First 1
[Environment]::SetEnvironmentVariable("JAVA_HOME", $jdk.FullName, "User")

# --- new terminal: verify ---
java -version        # openjdk version "21.0.x" ... Temurin
echo $env:JAVA_HOME  # C:\Program Files\Eclipse Adoptium\jdk-21...
```

If `java -version` *shows another version, an older Java comes first on PATH: run* `where.exe java` *and remove or move the older entry in System Properties → Environment Variables.*

Step 4 · Apache Maven 3.9

Download the **Binary zip archive** from maven.apache.org/download.cgi and unzip it into **C:\tools** (Maven 4 is still a release candidate — stay on 3.9.x). Then:

```
$mvn = "C:\tools\apache-maven-3.9.11"                # the folder you unzipped
[Environment]::SetEnvironmentVariable("MAVEN_HOME", $mvn, "User")
$p = [Environment]::GetEnvironmentVariable("Path", "User")
[Environment]::SetEnvironmentVariable("Path", "$p;$mvn\bin", "User")

# --- new terminal ---
mvn -v      # Apache Maven 3.9.x ... Java version: 21.0.x
```

Once the repo contains the Maven wrapper (mvnw.cmd, added in Sprint 0), any other laptop can build without Maven installed — the wrapper downloads the right version itself.

Step 5 · Docker Desktop

```
winget install --id Docker.DockerDesktop -e
# Restart (or sign out and in), then start Docker Desktop from the Start menu:
#   accept the terms, skip sign-in, wait for "Engine running"

# --- new terminal ---
docker version              # shows Client AND Server sections
docker compose version      # Docker Compose version v2.x
docker run --rm hello-world # prints "Hello from Docker!"
```

Recommended settings (Docker Desktop → Settings)

- General → **Use the WSL 2 based engine**: on (the default).
- General → **Start Docker Desktop when you sign in**: your choice — turning it off saves RAM on non-coding days.

- Resources → **Resource Saver**: on. It pauses Docker when no containers are running.
- Memory is controlled by your .wslconfig file, not by a slider (Section 9).

Step 6 · IntelliJ IDEA and VS Code

```
# JetBrains Toolbox - then install IntelliJ IDEA from the Toolbox window
winget install --id JetBrains.Toolbox -e
winget install --id Microsoft.VisualStudioCode -e

# --- new terminal: VS Code extensions used in this project ---
code --install-extension vscjava.vscode-java-pack
code --install-extension vmware.vscode-boot-dev-pack
code --install-extension humao.rest-client          # runs the .http request files
code --install-extension ms-azuretools.vscode-docker
code --install-extension redhat.vscode-yaml
code --install-extension editorconfig.editorconfig
code --install-extension dbaeumer.vscode-eslint     # frontend (Sprint 9+)
code --install-extension esbenp.prettier-vscode
code --install-extension bradlc.vscode-tailwindcss
```

Use IntelliJ for the Java services and VS Code for the React app, YAML and the .http request files. Section 8 covers the IntelliJ project settings.

Step 7 · Node.js LTS

```
winget install --id OpenJS.NodeJS.LTS -e

# --- new terminal ---
node -v     # v24.x today (Active LTS); Node 26 becomes the LTS at the end of October 2026
npm -v

# Later, to move to the newest LTS:
winget upgrade --id OpenJS.NodeJS.LTS
```

You first need Node in Sprint 9 (February 2027). Before that sprint, upgrade so you are on the Active LTS line (26).

Step 8 · Helper apps

```
winget install --id DBeaver.DBeaver.Community -e   # look inside PostgreSQL
winget install --id RedisInsight.RedisInsight -e   # look inside Redis
winget install --id Postman.Postman -e             # optional API client
```

Step 9 · AI and Kubernetes tools (when you reach them)

```
# Sprint 7 - Ollama (runs on your GPU if you have one)
winget install --id Ollama.Ollama -e
ollama pull nomic-embed-text      # embedding model, ~0.3 GB
ollama pull qwen3:8b              # chat model with tool calling, ~5 GB
#   16 GB RAM or less and no GPU? use: ollama pull qwen3:4b
ollama list
curl.exe http://localhost:11434/api/tags     # Ollama answers on port 11434

# Sprint 12 - local Kubernetes and load testing
winget install --id Kubernetes.kind -e
winget install --id Kubernetes.kubectl -e
winget install --id GrafanaLabs.k6 -e
```

On Windows the native Ollama app is the better choice: unlike the Docker version it can use your GPU. Don't run both — they share port 11434. A hosted API key (OpenAI, Gemini...) also works: set a spending limit before you use it.

5. Get the AuraMart code onto the laptop

Option 1 — the repo is already on GitHub (any laptop after the first)

```
New-Item -ItemType Directory -Force -Path C:\dev | Out-Null
cd C:\dev
gh repo clone <your-github-username>/auramart
cd auramart
```

That's all: the repo already contains the Maven wrapper, CI workflow and config files. Go to Section 7.

Option 2 — the very first laptop, starting from the starter code

```
# 1. Copy the starter code to C:\dev, then the setup kit's files on top of it
New-Item -ItemType Directory -Force -Path C:\dev | Out-Null
Copy-Item "$env:USERPROFILE\Downloads\AuraMart\3-phase1-starter\auramart" C:\dev -Recurse
Copy-Item "$env:USERPROFILE\Downloads\AuraMart\2-laptop-setup\*" C:\dev\auramart -Recurse -Force
cd C:\dev\auramart

# 2. Start Git and add the Maven wrapper
git init -b main
mvn -N wrapper:wrapper

# 3. First commit - mark mvnw as executable so it runs in Linux CI
git add .
git update-index --chmod=+x mvnw
git commit -m "chore: initial scaffold with Maven wrapper (AM-2)"

# 4. Create the GitHub repo and push
gh repo create auramart --public --source=. --remote=origin --push
```

The second Copy-Item also replaces the starter's docker-compose.yml with the kit's version (Kafka memory cap and opt-in profiles for later sprints). This is story AM-2 in the execution plan; the Phase 1 Build Guide (Steps 2 and 3) walks through it with a check at every step.

Protect the main branch (once)

1. GitHub → your repo → **Settings → Rules → Rulesets → New branch ruleset**.
2. Name it “main”, set Enforcement to **Active**, target the **default branch**.
3. Tick **Require a pull request before merging** and set required approvals to **0** — you can't approve your own pull request.
4. After CI has run once (push any commit), tick **Require status checks to pass** and add the **build** check.
5. Save. From now on every change goes through a pull request with a green build.

6. Download everything up front (optional)

On a slow or metered connection, download everything once on good Wi-Fi. Run this from **C:\dev\auramart** with Docker Desktop running:

```
# Container images + Java libraries
powershell -ExecutionPolicy Bypass -File .\scripts\predownload.ps1

# ...plus the AI models (~5 GB)
powershell -ExecutionPolicy Bypass -File .\scripts\predownload.ps1 -WithModels

# ...with the smaller chat model instead
powershell -ExecutionPolicy Bypass -File .\scripts\predownload.ps1 -WithModels -ChatModel qwen3:4b
```

The same, by hand:

```
# Every container image, including the later-sprint profiles
docker compose --profile mail --profile observability --profile security pull

# Java libraries and Maven plugins
.\mvnw.cmd -B dependency:go-offline

# AI models
ollama pull nomic-embed-text
ollama pull qwen3:8b
```

7. Start everything and verify

Start the infrastructure

```
cd C:\dev\auramart
docker compose up -d     # first time downloads ~1.2 GB of images
docker compose ps        # wait until postgres, redis and kafka show (healthy)
```

Check each part

Check	Command	Expected
PostgreSQL + pgvector	`docker exec auramart-postgres psql -U catalog_svc -d catalog_db -c "SELECT extname FROM pg_extension;"`	plpgsql, vector, pg_trgm
Redis	`docker exec auramart-redis redis-cli ping`	PONG
Kafka	`docker exec auramart-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list`	No error (empty list)
Kafka UI	`Open http://localhost:8090`	Cluster auramart-local, 1 broker
Catalog service	`.\mvnw.cmd -pl services/catalog-service spring-boot:run`	Started CatalogServiceApplication
Health	`curl.exe http://localhost:8081/actuator/health`	{"status":"UP", ...}
API	`curl.exe "http://localhost:8081/api/v1/products?size=3"`	JSON with 3 products
Swagger UI	`Open http://localhost:8081/swagger-ui.html`	All product endpoints listed
Tests	`.\mvnw.cmd test`	BUILD SUCCESS
Whole laptop	`powershell -ExecutionPolicy Bypass -File .\scripts\check-env.ps1`	All required checks passed

Use `curl.exe`, not `curl` — in Windows PowerShell 5.1, "curl" is an alias for a different command.

Kafka smoke test (send and read one message)

```
docker exec auramart-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 `
    --create --topic smoke-test --partitions 3 --replication-factor 1

docker exec -it auramart-kafka /opt/kafka/bin/kafka-console-producer.sh `
    --bootstrap-server localhost:9092 --topic smoke-test
#   type:  hello   then press Enter, then Ctrl+C

docker exec auramart-kafka /opt/kafka/bin/kafka-console-consumer.sh `
    --bootstrap-server localhost:9092 --topic smoke-test --from-beginning --max-messages 1
#   prints: hello

docker exec auramart-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 `
    --delete --topic smoke-test
```

Connect the GUI tools

Tool	Connection settings
DBeaver	New connection → PostgreSQL → Host localhost · Port 5432 · Database catalog_db · User catalog_svc · Password catalog_pw → Test connection (it downloads the driver)
Redis Insight	Add Redis database → Host localhost · Port 6379 · no password
Kafka UI	Nothing to set up — http://localhost:8090

These are local-development passwords from the Compose setup. Never reuse them anywhere real.

8. Configure the IDE, secrets and ports

IntelliJ IDEA project settings

1. **File → Open** → C:\dev\auramart → **Trust Project**. IntelliJ imports the Maven modules.
2. **File → Project Structure → Project**: SDK = Temurin 21, language level 21.
3. **Settings → Build, Execution, Deployment → Build Tools → Maven**: Maven home path = **Use Maven wrapper**.
4. Run **CatalogServiceApplication** once (green arrow) — IntelliJ creates a run configuration.
5. Edit that configuration → Modify options → **Add VM options**: `-Xmx384m` (keeps each service small; repeat for every service you add).
6. **Help → Change Memory Settings**: 2048 MB is plenty for this project.

Secrets and environment variables

- Copy `.env.example` to `.env` for local overrides. `.env` is in .gitignore — it never reaches GitHub.
- API keys for one terminal session: `$env:OPENAI_API_KEY = "sk-..."`. For good: Windows Settings → System → About → Advanced system settings → Environment Variables (user variables).
- In IntelliJ, put secrets in the run configuration's **Environment variables** field, not in application.yml.
- In GitHub Actions, use repo Settings → Secrets and variables → Actions.
- If a key ever lands in a commit, revoke it at the provider immediately — deleting the commit is not enough.

Every port AuraMart uses

Port	What	From
8080	API Gateway	S2
8081 · 8082 · 8083 · 8084	Catalog · Order · AI assistant · Notification services	S1–S8
5173	React dev server (Vite)	S9
5432	PostgreSQL	S0
6379	Redis	S0
9092	Kafka — for apps on Windows (containers use kafka:19092 internally)	S0
8090	Kafka UI	S0
11434	Ollama	S7
1025 · 8025	Mailpit SMTP · Mailpit inbox	S6
3000 · 4317 · 4318	Grafana · OpenTelemetry gRPC · OpenTelemetry HTTP	S11
8180	Keycloak (8080 is the gateway)	S12

Something else already using a port? The check script lists them all; or run `Get-NetTCPConnection -State Listen -LocalPort 5432`. Fix it by stopping that program or changing the left-hand number in docker-compose.yml (e.g. "5433:5432").

9. Memory and performance

Docker's Linux VM will take as much memory as it likes unless you cap it. Copy `infra\windows\wslconfig.example` to `C:\Users\<you>\.wslconfig`, pick your memory line, then run `wsl --shutdown` and restart Docker Desktop.

```
# C:\Users\<you>\.wslconfig
[wsl2]
memory=8GB
processors=4
swap=4GB

[experimental]
autoMemoryReclaim=gradual
```

Laptop RAM	memory= in .wslconfig	How to work
8 GB	4GB	Use a hosted LLM instead of Ollama; run only the 1–2 services you are working on; skip kind until you can borrow RAM by closing everything else
16 GB	8GB	Everything works; use qwen3:4b if there is no GPU; stop the Compose services while working on kind
32 GB	16GB	Run the whole system at once, including a bigger local model

What runs in each phase (rough memory use)

Sprints	Running	≈ RAM in use
S1–S3	Postgres, Redis, Kafka, Kafka UI + 1–2 services + IntelliJ	6–7 GB
S4–S6	+ Order and Notification services, Mailpit	8 GB
S7–S8	+ AI assistant + Ollama model (or hosted API)	12–14 GB (9 GB with a hosted API)
S9–S10	+ Vite dev server and a browser	10–14 GB
S11–S12	+ Grafana LGTM, Keycloak, kind cluster	14 GB+ — stop what you don't need

Rough figures — the check script and Task Manager show the real numbers on your laptop.

Speed tips

- Keep the project in C:\dev, never in OneDrive.
- Windows 11 **Dev Drive** (Settings → System → Storage → Advanced storage settings → Disks & volumes → Create Dev Drive) makes builds faster; it needs 50 GB of free space. Optional.
- Or exclude your code and caches from Defender scanning (Terminal (Admin)): `Add-MpPreference -ExclusionPath C:\dev, $env:USERPROFILE\.m2`. Only exclude folders you trust.
- Close Docker Desktop on days you don't code.

10. Working across two laptops

GitHub is the single source of truth. Anything committed and pushed follows you; everything else lives on one laptop only and is rebuilt on the other.

Thing	Where it lives	On the other laptop
Code, docs, ADRs, CI workflow	GitHub	git pull
Task board and issues	GitHub Projects	Nothing to do
Database data	A Docker volume on each laptop	Rebuilt by Flyway + seed data; docker compose down -v for a clean start
Kafka messages, Redis cache	Docker volumes	Not copied — they are test data
Secrets (.env, API keys)	Only on each laptop	Copy by hand (a password manager is ideal) — never through Git
IDE settings	Each laptop	IntelliJ: sign in to Backup and Sync. VS Code: turn on Settings Sync
AI models	Each laptop	ollama pull again (or predownload.ps1 -WithModels)
Maven / npm caches	Each laptop	Downloaded on the first build

The switching routine

```
# On the laptop you are leaving - push everything, even unfinished work
git switch -c feature/AM-26-outbox      # or stay on your current feature branch
git add -A
git commit -m "wip: outbox relay (AM-26)"
git push -u origin HEAD

# On the laptop you are moving to
cd C:\dev\auramart
git fetch
git switch feature/AM-26-outbox
git pull
docker compose up -d
```

“wip” commits are fine on a feature branch — the squash merge turns them into one clean commit on main. Never leave work only on one laptop: a lost or broken laptop then costs nothing.

11. Troubleshooting

Problem	Cause	Fix
Docker Desktop won't start / “WSL 2 is not installed”	WSL missing or out of date, or virtualization off	wsl --install --no-distribution, then wsl --update; restart. Check Task Manager → CPU → Virtualization: Enabled.
“You are not allowed to use Docker, you must be in the docker-users group”	Your account isn't in the group	Terminal (Admin): net localgroup docker-users $env:USERNAME /add — then sign out and back in.
“… is not recognized as the name of a cmdlet”	The terminal was open before the install	Close all terminals and open a new one.
“npm.ps1 cannot be loaded because running scripts is disabled”	PowerShell script policy	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
mvn -v shows Java 17 / “release version 21 not supported”	JAVA_HOME or PATH points to an older JDK	Redo Step 3; check where.exe java; open a new terminal.
“Ports are not available” / “port is already allocated”	Another program, or a Windows-reserved port range	Find it: Get-NetTCPConnection -State Listen -LocalPort <port>. Reserved ranges: netsh interface ipv4 show excludedportrange protocol=tcp. Change the host port in docker-compose.yml.
Postgres: “password authentication failed for catalog_svc” or database missing	The init script didn't run because the data volume already existed	docker compose down -v, then docker compose up -d (wipes local data).
App can't reach Kafka (mentions kafka:19092)	The app used the containers-only listener	Apps on Windows use localhost:9092; only containers use kafka:19092.
CI fails: “/usr/bin/env: 'sh\r'” or “./mvnw: Permission denied”	mvnw has Windows line endings or no execute bit	Keep the .gitattributes file; git add --renormalize . ; git update-index --chmod=+x mvnw ; commit and push.
“Filename too long” in git or npm	Windows 260-character path limit	Step 1 long-paths command; git config --global core.longpaths true; keep the repo in C:\dev.
Testcontainers: “Could not find a valid Docker environment”	Docker Desktop isn't running	Start Docker Desktop and wait for “Engine running”.
Maven “PKIX path building failed” or Docker pulls fail on office Wi-Fi	Company proxy / SSL inspection	Use a home network, or ask IT for the Maven mirror and proxy settings (Docker Desktop → Settings → Resources → Proxies).
Laptop freezes when Docker is running	WSL VM has no memory cap	Add .wslconfig (Section 9), wsl --shutdown, restart Docker Desktop.
Ollama answers very slowly	The model runs on the CPU	ollama ps shows CPU/GPU. Use qwen3:4b, update GPU drivers, or use a hosted API.
curl prints a strange table of properties	In Windows PowerShell 5.1 curl means Invoke-WebRequest	Use curl.exe, or PowerShell 7.

12. Daily cheat sheet

Start of the day

```
# Docker Desktop running? Then:
cd C:\dev\auramart
git switch main
git pull
docker compose up -d
docker compose ps
```

End of the day

```
git push                    # never leave work only on this laptop
docker compose stop         # stops containers, keeps data
```

Useful commands

```
docker compose logs -f kafka               # follow one service's log
docker compose restart redis               # restart one service
docker compose --profile mail up -d        # add Mailpit (Sprint 6+)
docker compose down -v                     # WIPE all local data and start clean
docker compose pull                        # update images
docker system df                           # disk used by Docker
docker system prune                        # remove stopped containers and dangling images
wsl --shutdown                             # free all Docker/WSL memory (stops Docker)
.\mvnw.cmd -pl services/catalog-service spring-boot:run
.\mvnw.cmd test
powershell -ExecutionPolicy Bypass -File .\scripts\check-env.ps1
```

Keeping tools up to date (once a month)

```
winget upgrade                  # list available updates
winget upgrade --all            # install them (Terminal (Admin))
wsl --update
```

Appendix A. macOS and Linux equivalents

For reference if you ever set up a Mac or a Linux laptop. Everything after installation (Sections 5–12) works the same, using ./mvnw instead of .\mvnw.cmd.

Tool	macOS (Homebrew)	Ubuntu / Debian
Git	`brew install git`	`sudo apt install git`
GitHub CLI	`brew install gh`	`apt repository from cli.github.com`
Java 21	`brew install --cask temurin@21`	`sudo apt install openjdk-21-jdk`
Maven	`brew install maven`	`sudo apt install maven`
Docker	`Docker Desktop installer from docker.com (or OrbStack)`	`Docker Engine from docs.docker.com (apt repository)`
Node.js LTS	`brew install node@24`	`nvm or NodeSource`
IntelliJ / VS Code	`JetBrains Toolbox; brew install --cask visual-studio-code`	`JetBrains Toolbox; code.visualstudio.com .deb`
Ollama	`App from ollama.com (runs on the Apple GPU — don't use the Docker version on a Mac)`	`curl -fsSL https://ollama.com/install.sh | sh`
kind / kubectl / k6	`brew install kind kubectl k6`	`Release binaries from each project's site`

Appendix B. What's in the 2-laptop-setup folder

File	What it does
`scripts\README.txt`	Quick start for this kit: the same commands as Section 3
`scripts\install-tools.ps1`	Installs the whole tech stack with winget, Maven, JAVA_HOME, WSL 2, long paths, Git settings, VS Code extensions
`scripts\check-env.ps1`	Checks versions, Docker, script policy, ports and running containers
`scripts\predownload.ps1`	Pulls every container image, the Java libraries and (optionally) the AI models
`docker-compose.yml`	Phase 1 infrastructure plus opt-in profiles: mail, observability, security, ai
`.gitattributes · .editorconfig`	Same line endings and formatting on Windows and in Linux CI
`.github\workflows\ci.yml`	GitHub Actions: build and test on every push and pull request
`.github\pull_request_template.md`	The Definition of Done checklist on every pull request
`infra\windows\wslconfig.example`	Memory limit for Docker / WSL 2 (Section 9)
`infra\k8s\metrics-server-patch.json`	Lets autoscaling work on kind (Sprint 12)

The CI workflow, for reference

.github/workflows/ci.yml

```
# Build and test every push to main and every pull request.
# Runs on Linux, so ./mvnw must have LF line endings and the execute bit (see the setup guide).
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v5

      - name: Set up JDK 21
        uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '21'
          cache: maven

      - name: Build and test
        run: ./mvnw -B verify
```